

TP DevOps Final — Résumé des 5 parties

Étudiants : Yanis GONTO, ADOU Jean Paul, KONE Yah Houriya

Dépôt GitHub : <https://github.com/haydenprod/devops-tp-final>

Date : Mai 2026

Partie 1 — Préparation de l'infrastructure (/20)

Objectif

Automatiser le déploiement d'une VM Debian sur VirtualBox avec k3s (Kubernetes léger) pré-installé.

Ce qui a été réalisé

- **Vagrantfile** créé avec `debian/bookworm64`, IP fixe `192.168.56.10`, 2 Go RAM, 2 vCPU.
- **Provisioning automatique** : `curl` installé, `k3s` installé via `curl -sfL https://get.k3s.io | sh -`, idempotent (ne réinstalle pas si déjà présent).
- **kubeconfig** rendu lisible sans `sudo` : `chmod 644 /etc/rancher/k3s/k3s.yaml`.
- VM lancée avec `vagrant up`, accessible via `vagrant ssh k3s`.

Stack technique

Outil	Version / Détail
Vagrant	Dernier stable
VirtualBox	Installé sur l'hôte
Debian	bookworm64
k3s	Dernière version stable
IP VM	192.168.56.10

Partie 2 — Conteneurisation de l'application (/10)

Objectif

Récupérer le code source de l'API Node.js / MySQL et créer une image Docker optimisée.

Ce qui a été réalisé

- Code source cloné depuis github.com/almoggutin/Node-Express-REST-API-MySQL-JS-Example dans `api-lacets/`.
- **Dockerfile** multi-stage (base `node:alpine`) pour minimiser la taille de l'image.
- Image construite localement et poussée sur **Docker Hub** (`haydenprod/lacets-api:latest`).
- Variables de connexion MySQL externalisées via variables d'environnement.

Résultat

Image Docker légère (base alpine), prête à être déployée sur k3s.

Partie 3 — Déploiement sur Kubernetes (/10)

Objectif

Créer les manifests Kubernetes pour déployer l'API et sa base de données MySQL dans k3s.

Ce qui a été réalisé

- `k8s/mysql.yaml` : Deployment MySQL + Service ClusterIP (interne au cluster) + PersistentVolumeClaim pour la persistance des données.
- `k8s/api.yaml` : Deployment de l'API avec `HorizontalPodAutoscaler` (min 1 pod, max 3) basé sur CPU/mémoire, Service ClusterIP (API joignable uniquement depuis l'intérieur du cluster).
- Données MySQL persistantes via PVC.
- Scaling automatique : minimum 1 pod, jusqu'à 3 en pic de charge.

Fichiers

Fichier	Contenu
<code>k8s/mysql.yaml</code>	Deployment + Service + PVC MySQL
<code>k8s/api.yaml</code>	Deployment + HPA + Service API

Partie 4 — CI/CD avec GitHub Actions (/20)

Objectif

Pipeline CI/CD déclenchée sur chaque push sur `main`, incluant build, déploiement et runner self-hosted.

Ce qui a été réalisé

- **Workflow** `.github/workflows/cicd.yml` créé avec les étapes :
 1. Checkout du dépôt
 2. Installation de Docker et kubectl
 3. Configuration de l'accès au cluster (kubecfg)
 4. **Build de l'image Docker** ■ (étape validée)
 5. **Déploiement sur k3s** via `kubectl apply -f k8s/`
 6. Vérification des pods
- **Runner self-hosted** installé dans la VM `k3s-node` (binaire v2.334.0).
- ■■ **Point bloquant** : Erreur de token OAuth expiré côté GitHub (`The token expired on 05/07/2026 03:49:00`) — problème identifié dans les logs `_diag`, non lié à la configuration mais à un bug de session GitHub Actions.
- Pipeline fonctionnelle sur `ubuntu-latest` jusqu'à l'étape `kubectl`, qui échoue car le cluster est en `127.0.0.1` (inaccessible depuis un runner hébergé par GitHub).

Architecture CI/CD

push sur main



GitHub Actions



- Build image Docker (■ validé)
- Push vers registry
- kubectl apply (nécessite runner self-hosted)

Partie 5 — Monitoring et observabilité (/20)

Objectif

Déployer Prometheus + Grafana sur une VM dédiée, avec node_exporter sur les deux VMs, et afficher le dashboard 1860.

Ce qui a été réalisé

- Vagrantfile mis à jour avec une seconde VM `monitoring` (IP 192.168.56.20, 2 Go RAM).
- node_exporter v1.8.2 installé et activé via systemd sur :

- k3s-node (192.168.56.10:9100) ■

- monitoring (192.168.56.20:9100) ■

- Prometheus v2.53.0 installé sur `monitoring`, service systemd actif, configuration :

- job k3s-node → 192.168.56.10:9100

- job monitoring → 192.168.56.20:9100

- Grafana 11.0.0 installé sur `monitoring` via `.deb` (dépendance `musl` résolue manuellement), service actif sur port 3000.
- Dashboard accessible sur `http://192.168.56.20:3000`.
- Dashboard 1860 (Node Exporter Full) à importer depuis Grafana UI.

Ports et accès

Service	VM	Adresse
node_exporter	k3s-node	http://192.168.56.10:9100
node_exporter	monitoring	http://192.168.56.20:9100
Prometheus	monitoring	http://192.168.56.20:9090
Grafana	monitoring	http://192.168.56.20:3000